CODITY.AI TECH ASSIGNMENT SUBMISSION

Production-Grade Distributed Job Scheduling Platform

Deep-dive Architecture, Relational Schema Normalization, and Resiliency Verification.

SUBMISSION INFORMATION

Registration Number:	RA2311026010746
Candidate Name:	Prudhvi
Project Platform:	Distributed Job Scheduler (Overdrive)
GitHub Repository:	https://github.com/prudhvi98491/distributed-job-scheduler
Date of Submission:	July 5, 2026
Academic Institution:	SRM Institute of Science and Technology

1. Executive Summary & Design Philosophy

Modern distributed backend architectures rely heavily on background execution engines to offload I/O-intensive, computationally heavy, or delayed tasks. Building a scheduler that guarantees exactly-once execution semantics, honors queue concurrency limits, handles failovers, and operates transparently requires robust relational models and transaction-level safety locks.

This project, named 'Overdrive', is designed from first-principles backend engineering patterns. It employs FastAPI for async API handling and SQLAlchemy combined with aiosqlite for non-blocking relational operations. By implementing a Write-Ahead Logging (WAL) configuration on SQLite, the system enjoys concurrent read capabilities and low-latency database access, which allows multiple workers to poll the same database file in parallel without lockouts.

Core Architecture Goals

- **Atomicity:** Jobs are claimed by workers using transactional locks, preventing double-claiming.
- **Queue Limits:** Strictly enforces concurrency limits at the queue level to prevent resource starvation.
- **Fault Tolerance:** Orphaned jobs are automatically reclaimed if a worker crashes or goes offline.
- **Workflows:** Sequential job dependencies are blocked until parents successfully complete.
- **Observability:** Complete log tracing and throughput stats visible in real-time on the UI.

2. Relational Database & Normalization Theory

A relational database design requires adhering to Normalization forms to prevent insertion, update, and deletion anomalies. Overdrive achieves Third Normal Form (3NF) across all entities:

1. First Normal Form (1NF): All attributes contain atomic values. Payloads are represented in a structured text column (JSON) to allow variable payload structures without violating column atomicity.
2. Second Normal Form (2NF): The tables are in 1NF and all non-key attributes are fully functionally dependent on the primary keys. We enforce single-column primary keys (uuid for jobs, auto-increment integer for others).
3. Third Normal Form (3NF): No transitive dependencies exist. For example, a job is associated with a queue, and a queue is associated with a project. Jobs do not hold projects directly; instead, they traverse relations, preventing redundant updates if a queue moves between projects.

Table Name	Schema Purpose & Normalization Details
users	Stores credentials (hashed via bcrypt), roles (admin, user, viewer), and metadata.
organizations	Enables multi-tenancy, grouping projects and user access levels.
projects	Contains project workspaces owned by organizations.
queues	Defines job queues with priority, concurrency_limit, and retry policies.
jobs	Core queue table containing status (queued, running, completed, failed, dlq), payload, worker_id, and parent_job_id (workflows).
job_executions	Tracks execution details, duration (ms), worker, and trace errors.
workers	Worker fleet registry monitoring hostnames, IP addresses, heartbeats, and current loads.
cron_jobs	Schedules recurring cron jobs to dynamically queue new jobs on intervals.
dead_letter_queue	Holds permanently failed tasks for isolation and manual replay.

3. Atomic Claiming & Concurrency Engineering

In a distributed scheduler, multiple worker threads query the database for pending jobs. If two workers claim the same job simultaneously, it violates exactly-once execution. To ensure thread safety, the claim logic utilizes an atomic SELECT-and-UPDATE transaction:

1. Active Queues: First, we select all queues that are not paused and whose active running jobs are strictly less than their configured concurrency limits.
2. Priority Matching: We query candidate jobs from these queues, sorting by priority (queue priority + job priority override) in descending order, then by creation date.
3. Transaction Lock: This selection and update is executed within a database transaction with IMMEDIATE write locks, preventing database access conflicts.

Database Locking Mechanics (Code Concept)

```
async with db.begin():
    # 1. Check active queues under concurrency limits
    eligible_queue_ids = [q.id for q in queues if active_cnt(q.id) < q.concurrency]

    # 2. Query and claim next job atomically
    job = await db.execute(select(Job)
        .filter(Job.status == 'queued', Job.queue_id.in_(eligible_queue_ids))
        .order_by(Job.priority_override.desc(), Job.created_at.asc())
        .limit(1))
    if job:
        job.status = 'claimed'; job.worker_id = worker_id
        await db.commit()
```

4. Worker Resiliency & Failover Mechanics

Distributed environments must handle node crashes and connectivity issues. Overdrive implements a robust heartbeating and orphan recovery cycle:

1. Heartbeats: Active workers update their 'last_heartbeat_at' timestamp in the database every 5 seconds.
2. Offline Detection: A background task on each worker periodically queries the database for worker records whose heartbeat is older than 15 seconds. If a worker goes offline, it is marked as offline.
3. Failover Recovery: Any job currently marked as 'running' or 'claimed' on the offline worker is automatically re-queued. The job status is set back to 'queued', worker_id is cleared, and an error message ('Recovered from offline worker') is logged. This ensures no jobs are lost or left in a stuck state.
4. Graceful Shutdown: When a worker receives a SIGINT/SIGTERM signal, it stops claiming new jobs, completes active tasks, and releases claims on others before exiting.

Workflow Sequential Dependencies Engine

Overdrive supports DAG-like sequential workflows. When a job is dispatched with a 'parent_job_id' specified, the engine checks if the parent job has completed. If the parent is still running or queued, the child job is created with a 'blocked' status. When a job completes successfully, the worker queries the database for any jobs blocked on it and updates their status to 'queued' to kick off the next stage of the pipeline automatically.

5. REST API Documentation

Method	Endpoint	Description / Functionality
POST	/api/auth/register	Registers a user account with hashed credentials.
POST	/api/auth/login	Authenticates credentials and issues a JWT token.
POST	/api/queues	Creates a queue specifying concurrency limit & priority.
PATCH	/api/queues/{id}	Updates queue configs or pauses/resumes queue state.
POST	/api/jobs	Enqueue single task (immediate, delayed, or workflow).
POST	/api/jobs/batch	Dispatches bulk job payloads under a single API call.
GET	/api/jobs	Lists jobs with pagination, queue, and status filters.
POST	/api/jobs/{id}/retry	Manually replays a failed or DLQ job.
GET	/api/metrics	Returns aggregate metrics, throughput trends, and error counts.

Schema Structures (Pydantic)

All incoming payloads are strictly validated using Pydantic schemas. For example, when creating a job, the client sends a 'JobCreate' payload containing 'queue_name' (string), 'name' (string), 'payload' (optional dictionary), 'priority_override' (optional integer), and 'parent_job_id' (optional string). Pydantic automatically validates these types and returns structured error responses if the data is invalid.

6. Testing & Technical Verification

We implemented comprehensive integration tests using pytest and pytest-asyncio to verify the scheduler's behavior under different conditions:

1. Database Seeding: Verifies that primary/foreign key relationships work correctly.
2. Retries & Policies: Simulates a job execution failure and verifies that it is rescheduled according to the retry policy (re-queued with backoff delay).
3. Workflow unblocking: Enqueues a parent and child job. Verifies that the child job remains blocked until the parent completes, and then successfully transitions to queued.

Pytest Output Logs

```
tests/test_scheduler.py::test_database_seeding_and_relations PASSED
tests/test_scheduler.py::test_job_execution_lifecycle_and_retries PASSED
tests/test_scheduler.py::test_workflow_dependencies_unblocking PASSED
============================== 3 passed in 2.02s ===============================
```

Verification GitHub Repository Link

https://github.com/prudhvi98491/distributed-job-scheduler